

# AFRILANG Stdlib

## std/c/statchi2

Bibliothèque AFRILANG `std/c/statchi2` (niveau complex, catégorie complex). Elle expose 6 fonction(s) :  
chi2Stat, cramer

```
`import "std/c/statchi2" . `use statchi2`
```

- `chi2Stat(observed list number, expected list number) ? number`
- `cramersV(chi2 number, n number, k number) ? number`
- `expectedUniform(total number, bins number) ? list number`
- `goodnessOfFit(observed list number, expected list number) ? number`
- `residuals(observed list number, expected list number) ? list number`
- `df(bins number) ? number`

Category: complex | Tier: complex | Functions: 6

# FRANCAIS

## 1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/statchi2` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : chi2Stat, cramer

```
`import "std/c/statchi2" . `use statchi2`  
- `chi2Stat(observed list number, expected list number) ? number`  
- `cramersV(chi2 number, n number, k number) ? number`  
- `expectedUniform(total number, bins number) ? list number`  
- `goodnessOfFit(observed list number, expected list number) ? number`  
- `residuals(observed list number, expected list number) ? list number`  
- `df(bins number) ? number`
```

## 2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/statchi2"  
use statchi2
```

Niveau : complex · Catégorie : Modules complex (c/)

Domaines avancés ? graphes, NLP, réseaux complexes.

## 3. Référence API

```
? chi2Stat(observed list number, expected list number) ? number  
? crammersV(chi2 number, n number, k number) ? number  
? expectedUniform(total number, bins number) ? list  
? goodnessOfFit(observed list number, expected list number) ? number  
? residuals(observed list number, expected list number) ? list  
? df(bins number) ? number
```

## 4. Exemples d'utilisation

```
import "std/c/statchi2"  
use statchi2  
  
create result = chi2Stat(/* args */)   
say result  
  
create result = crammersV(/* args */)   
say result  
  
create result = expectedUniform(/* args */)   
say result  
  
create result = goodnessOfFit(/* args */)   
say result  
  
create result = residuals(/* args */)   
say result  
  
create result = df(/* args */)   
say result
```

## 5. Bonnes pratiques

```
? Préférez `import "std/c/statchi2"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

## 6. Annexe ? source

```
module statchi2  
  
function chi2Stat(observed list number, expected list number) returns number  
end
```

```
function cramersV(chi2 number, n number, k number) returns number
end

function expectedUniform(total number, bins number) returns list number
end

function goodnessOfFit(observed list number, expected list number) returns number
end

function residuals(observed list number, expected list number) returns list number
end

function df(bins number) returns number
end
end
```

# ENGLISH

## 1. Overview

AFRILANG library `std/c/statchi2` (tier complex, category complex). Exports 6 function(s):  
chi2Stat, cramersV, expectedU

```
`import "std/c/statchi2"` . `use statchi2`  
- `chi2Stat(observed list number, expected list number) ? number`  
- `cramersV(chi2 number, n number, k number) ? number`  
- `expectedUniform(total number, bins number) ? list number`  
- `goodnessOfFit(observed list number, expected list number) ? number`  
- `residuals(observed list number, expected list number) ? list number`  
- `df(bins number) ? number`
```

## 2. Installation & import

Add the module to your program:

```
import "std/c/statchi2"  
use statchi2
```

Tier: complex · Category: Complex modules (c/)  
Advanced domains ? graphs, NLP, complex networks.

## 3. API reference

```
? chi2Stat(observed list number, expected list number) ? number  
? cramersV(chi2 number, n number, k number) ? number  
? expectedUniform(total number, bins number) ? list  
? goodnessOfFit(observed list number, expected list number) ? number  
? residuals(observed list number, expected list number) ? list  
? df(bins number) ? number
```

## 4. Usage examples

```
import "std/c/statchi2"  
use statchi2  
  
create result = chi2Stat(/* args */)   
say result  
  
create result = cramersV(/* args */)   
say result  
  
create result = expectedUniform(/* args */)   
say result  
  
create result = goodnessOfFit(/* args */)   
say result  
  
create result = residuals(/* args */)   
say result  
  
create result = df(/* args */)   
say result
```

## 5. Best practices

```
? Prefer `import "std/c/statchi2"` at the top of your file.  
? Run `afirang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

## 6. Appendix ? source

```
module statchi2  
  
function chi2Stat(observed list number, expected list number) returns number  
end
```

```
function cramersV(chi2 number, n number, k number) returns number
end

function expectedUniform(total number, bins number) returns list number
end

function goodnessOfFit(observed list number, expected list number) returns number
end

function residuals(observed list number, expected list number) returns list number
end

function df(bins number) returns number
end
end
```